



What the Module Adds to Your Cluster

The same cluster is 30 resources hand-written and 57 through the module. The networking is identical, so the difference is 27 cluster-side resources, and 14 of them are security groups and their rules.

IN THIS LECTURE

01 The counts

30 against 57, and the 19 that are identical on both sides.

02 The 27 extras

Sorted by what each one is actually for.

03 Security groups and the launch template

The two largest groups, read line by line.

04 What was already there

The resources EKS created for both clusters.



THE COUNTS

The same working cluster, described by 30 resources or by 57

Both figures come from a state list on two running clusters. Both have two nodes Ready, so they are equally working clusters, and one of them is described by nearly twice as many objects.



Hand-written

30 managed resources

Every object was declared in a resource block you wrote and reviewed yourself.



Module

57 managed resources

The same cluster, plus every resource the module declares on your behalf.



zsh

```
$ terraform state list | wc -l    # hand-written
30

$ terraform state list | wc -l    # module
57

$ kubectl get nodes                # on both clusters
2 nodes    Ready
```



The extra 27 are not waste

They exist in the account, several of them bill, and several change the cluster security posture. The totals are still the wrong place to stop: a large part of both numbers is identical.

SORTED BY PURPOSE

14 of the 27 are security groups and their rules

The resources the module adds, grouped by what each one is for. The largest group is bigger than the other six combined.

Purpose	Count	Resources
 Network isolation	14	Cluster security group, node security group, 12 rules across the two
 Secrets encryption	4	KMS key, alias, IAM policy, policy attachment
 Tagging and orchestration	3	Two tags on the primary security group, one timer before the node group
 Node boot	2	Launch template, cluster service CIDR validator
 Access management	2	Access entry, access policy association
 Control-plane logging	1	CloudWatch log group
 Workload identity	1	OIDC provider



The module writes your network isolation

That is the useful description of it, rather than saying it doubles your resource count. Fourteen of twenty-seven is more than all the other purposes combined.

The hand-written cluster owns no security group

It uses the one EKS creates, where resources carrying that group are broadly permitted to reach each other. The module replaces that with two groups it owns and twelve rules.



Hand-written

0 groups, 0 rules

0

No security group appears anywhere in your state.

!

Traffic works because members of the EKS-created cluster group may broadly reach each other.

!

Nothing to review in a plan, nothing to tighten in code.



Module

2 groups, 12 rules

7

Node group rules admitting the control plane: 443 , the kubelet port, and five webhook ports individually.

3

Node to node: ephemeral ports, and CoreDNS twice, once for TCP and once for UDP.

1

One egress rule allowing outbound traffic from the nodes.

1

The twelfth sits on the cluster group and lets nodes reach the API server on 443.

443

10250 kubelet

4443

6443

8443

9443

10251

53 tcp + udp

ephemeral

THE REMAINING GROUPS

Four capabilities, two of them billing continuously

Each of the remaining groups adds a capability rather than wiring. The KMS key and the log group carry a charge for as long as they exist.

Capability	Resources	What it carries	Bills
 Secrets encryption	KMS key, alias, IAM policy, attachment	Secrets encrypted at rest under a key you control, at a monthly key charge plus per-request charges	continuously
 Control-plane logging	CloudWatch log group	Ingestion and storage of API server, authenticator and audit logs at 90-day retention	continuously
 Access management	Access entry, policy association	The same cluster-admin grant a single boolean gave you, now readable and adjustable	no
 Workload identity	OIDC provider	A standing trust relationship between the account and the cluster	no

 **None of the four was named anywhere in the module call**
Each is a capability a careful team adds eventually, arriving here already decided and already running.